

Code :

```
#include<iostream>
```

```
#include<queue>
```

```
#include<omp.h>
```

```
using namespace std;
```

```
class node {
```

```
public:
```

```
    int data;
```

```
    node *left, *right;
```

```
};
```

```
node* insert(node* root, int data) {
```

```
    if (!root) {
```

```
        root = new node{data, NULL, NULL};
```

```
        return root;
```

```
    }
```

```
    queue<node*> q;
```

```
    q.push(root);
```

```
    while (!q.empty()) {
```

```
        node* temp = q.front(); q.pop();
```

```
        if (!temp->left) {
```

```
            temp->left = new node{data, NULL, NULL};
```

```
            return root;
```

```
        } else q.push(temp->left);
```

```
        if (!temp->right) {
```

```
            temp->right = new node{data, NULL, NULL};
```

```
            return root;
```

```

        } else q.push(temp->right);
    }
    return root;
}

```

```

void bfs(node* root) {
    queue<node*> q;
    q.push(root);

```

```

    while (!q.empty()) {
        int size = q.size();

```

```

        #pragma omp parallel for
        for (int i = 0; i < size; i++) {
            node* curr;

```

```

            #pragma omp critical
            {
                curr = q.front();
                q.pop();
                cout << curr->data << " ";
            }

```

```

            #pragma omp critical
            {
                if (curr->left) q.push(curr->left);
                if (curr->right) q.push(curr->right);
            }
        }
    }
}

```

```

int main() {

```

```

node* root = NULL;

int data;

char ch;

do {

    cout << "Enter data: ";

    cin >> data;

    root = insert(root, data);

    cout << "Add more? (y/n): ";

    cin >> ch;

} while (ch == 'y');

cout << "BFS Traversal: ";

bfs(root);

}

```

Output:

```

PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA89AE87ADCE96EB3
\LP5\LP5> cd "c:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA89AE87ADCE96EB3
\2026-16\LP5\LP5\" ; if ($?) { g++ 1A.cpp -o 1A } ; if ($?) { .\1A }
Enter data: 5
Add more? (y/n): y
Enter data: 3
Add more? (y/n): y
Enter data: 2
Add more? (y/n): y
Enter data: 1
Add more? (y/n): y
Enter data: 7
Add more? (y/n): y
Enter data: 8
Add more? (y/n): n
BFS Traversal: 5 3 2 1 7 8
PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA89AE87ADCE96EB3
\LP5\LP5> 

```

Code:

```
#include <iostream>

#include <vector>

#include <stack>

#include <omp.h>

using namespace std;

vector<vector<int>> graph;

vector<bool> visited;

void dfs(int start) {
    stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int node = s.top();
        s.pop();

        if (!visited[node]) {
            visited[node] = true;
            cout << node << " ";

            #pragma omp parallel for
            for (int i = 0; i < graph[node].size(); i++) {
                int adj = graph[node][i];
                if (!visited[adj]) {
                    #pragma omp critical
                    s.push(adj);
                }
            }
        }
    }
}
```

```

}

int main() {
    int n, m, start;

    cout << "Enter nodes, edges, start: ";

    cin >> n >> m >> start;

    graph.resize(n);
    visited.resize(n, false);

    cout << "Enter edges:\n";
    for (int i = 0; i < m; i++) {
        int u, v;

        cin >> u >> v;

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    cout << "DFS: ";

    dfs(start);
}

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA89AE8
● \LP5\LP5> cd "c:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8
ers\2026-16\LP5\LP5\" ; if ($?) { g++ 1B.cpp -o 1B } ; if ($?) { .\1B }
Enter nodes, edges, start: 6 5 0
Enter edges:
0 1
0 2
1 3
1 4
2 5
DFS: 0 2 5 1 4 3
○ PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA89AE8
\LP5\LP5> 

```

Code:

```
#include<iostream>

#include<omp.h>

using namespace std;

void bubble(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        int first = i % 2;

        #pragma omp parallel for
        for (int j = first; j < n-1; j += 2) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;
    int arr[n];
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    bubble(arr, n);
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```

Output:

```
● \LP5\LP5> cd "c:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\se  
fers\2026-16\LP5\LP5\" ; if ($?) { g++ 2A.cpp -o 2A } ; if ($?) { .\2A }  
Enter size: 5  
Enter elements:  
5 3 4 1 2  
Sorted array:  
1 2 3 4 5  
○ PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8/  
\LP5\LP5> █
```

Code:

```
#include <iostream>

#include <omp.h>

using namespace std;

int main() {

    int n;

    cout << "Enter size: ";

    cin >> n;

    int arr[n];

    cout << "Enter elements:\n";

    for (int i = 0; i < n; i++)

        cin >> arr[i];

    int sum = 0, min_val = arr[0], max_val = arr[0];

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < n; i++)

        sum += arr[i];

    #pragma omp parallel for reduction(min:min_val)
    for (int i = 0; i < n; i++)

        if (arr[i] < min_val) min_val = arr[i];

    #pragma omp parallel for reduction(max:max_val)
    for (int i = 0; i < n; i++)

        if (arr[i] > max_val) max_val = arr[i];

    float avg = (float)sum / n;

    cout << "Sum: " << sum << endl;
```

```
cout << "Min: " << min_val << endl;

cout << "Max: " << max_val << endl;

cout << "Avg: " << avg << endl;

}
```

Output:

```
● \LP5\LP5> cd "c:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA8...
fers\2026-16\LP5\LP5\" ; if ($?) { g++ 3.cpp -o 3 } ; if ($?) { .\3 }
Enter size: 5
Enter elements:
8 9 2 7 1
Sum: 27
Min: 1
Max: 9
Avg: 5.4
○ PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA8...
\LP5\LP5> 
```

Code:

```
#include <iostream>

#include <omp.h>

using namespace std;

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            mergeSort(arr, l, m);
```

```

        #pragma omp section

        mergeSort(arr, m + 1, r);
    }

    merge(arr, l, m, r);
}
}

```

```

int main() {

    int n;

    cout << "Enter size: ";

    cin >> n;

    int arr[n];

    cout << "Enter elements:\n";

    for (int i = 0; i < n; i++)

        cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array:\n";

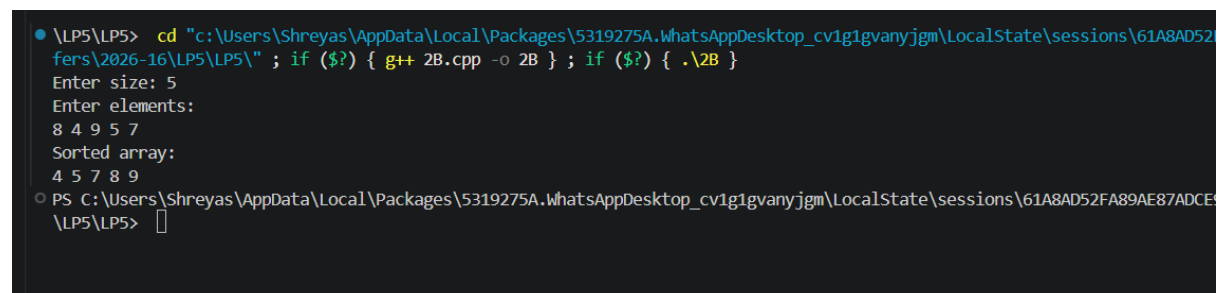
    for (int i = 0; i < n; i++)

        cout << arr[i] << " ";

}

```

Output:



```

● \LP5\LP5> cd "c:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52...
fers\2026-16\LP5\LP5\" ; if ($?) { g++ 2B.cpp -o 2B } ; if ($?) { .\2B }
Enter size: 5
Enter elements:
8 4 9 5 7
Sorted array:
4 5 7 8 9
○ PS C:\Users\Shreyas\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\61A8AD52FA89AE87ADCE...
\LP5\LP5>

```

```
%%writefile parallel_bfs.cpp
```

```
# Compile the code
```

```
!g++ -fopenmp parallel_bfs.cpp -o parallel_bfs
```

```
# Set the number of threads (optional)
```

```
%env OMP_NUM_THREADS=4
```

```
# Run the program
```

```
!./parallel_bfs
```